

5.1 Fix My Code

Understanding Exit Status Codes

Suppose we want to display the full path of the current directory using Python. We can achieve this by calling CLI commands directly from Python.

To start, we need to import the `os` module, which provides functions for interacting with the operating system.

```
In [1]: import os
```

We'll use Python's `os.system()` function and pass the `pwd` command to display the current directory path.

```
In [2]: result = os.system("pwd")
```

/app

After the command is executed, the current working directory is displayed.

```
In [3]: print(f"Exit status: {result}")
```

Exit status: 0

Since the `pwd` (print working directory) command is valid, it executes successfully. It returns an exit status code of 0, indicating the command ran without errors.

```
In [4]: result = os.system("pqd")
```

sh: 1: pqd: not found

Since `pqd` is not a valid system command (it's a typo of `pwd`), the command fails to execute. An error message `sh: 1: pqd: not found` is displayed. Notice that Python does not raise an exception in this case, this is a different type of error than a typical Python exception.

```
In [5]: print(f"Exit status: {result}")
```

Exit status: 32512

A non-zero exit status code is returned, indicating an error. Any non-zero exit status signals that the command didn't run successfully. It's crucial to pay attention to this feedback and change your code to run successfully.

In this case, change `pqd` to `pwd`.

```
In [6]: result = os.system("pwd")
        print(f"Exit status: {result}")
```

```
/app
Exit status: 0
```

Now it's your turn to fix incorrect command line code. You can tell the code is incorrect because it doesn't perform the intended action and returns a non-zero exit status code.

Task 5.1.1: Modify the code to display the contents of the current directory.

```
In [8]: result = os.system("ls")
        print(f"Exit status: {result}")
```

```
051-fix-my-code.ipynb
worldquant.txt
Exit status: 0
```

Fixing Permission Errors

We often have to interact with files and directories, but permission errors can sometimes prevent this. Permission errors occur when your Python script lacks the necessary rights to read, write, or execute files in specific locations.

Below is an example of command line code that successfully appends text to a file because it has the correct permissions.

```
In [10]: result = os.system("echo 'Hello, World! 🌍' > hello_world.txt")
        print(f"Exit status: {result}")
```

```
Exit status: 0
```

Let's change the file permissions to make the file read-only. The command only display a non-zero exit status code, indicating that it successfully changed the permissions.

```
In [11]: os.system("chmod 444 hello_world.txt")
```

```
Out[11]: 0
```

Now when we attempt to modify the file, it won't work because the necessary permissions are denied. This is also verified by the non-zero exit status code.

```
In [12]: os.system("echo 'Good night, moon! 🌙' >> hello_world.txt")
```

```
sh: 1: cannot create hello_world.txt: Permission denied
```

```
Out[12]: 512
```

We can change the file permissions to grant the necessary rights to perform the action, allowing us to run the commands successfully.

```
In [13]: os.system("chmod 666 hello_world.txt")
os.system("echo 'Good night, moon! 🌙' >> hello_world.txt")
```

```
Out[13]: 0
```

Here's another example of file permission issues.

```
In [14]: # Remove file
os.system("rm -f 'application.log'")
# Create new file
os.system("touch 'application.log'")
# Append entry to log file
os.system("echo 'Log entry 1' >> application.log")
# Append another entry to log file
os.system("echo 'Log entry 2' >> application.log")
# Display file contents
os.system("cat application.log")
# Change permissions to remove read access
os.system("chmod 000 application.log")
# Try to display file contents
os.system("cat application.log")
```

```
Log entry 1
```

```
Log entry 2
```

```
cat: application.log: Permission denied
```

```
Out[14]: 256
```

Task 5.1.2: Change the permissions of `application.log` to allow read access, use the `os.system` command.

```
In [15]: os.system("chmod 600 application.log")

# Display the contents of the file
os.system("cat application.log")
```

```
Log entry 1
```

```
Log entry 2
```

```
Out[15]: 0
```

Capturing Output of System Commands

`os.system` is a useful choice for commands that don't require using any of the resulting output. However, sometimes it is helpful to capture the output of a command for later use. If you want to use the output later, `subprocess.run` is a better choice.

We can use `subprocess.run` to capture the output of `ls`.

```
In [39]: import subprocess

result = subprocess.run(["ls"], capture_output=True, text=True)
```

```
print("Output of 'ls':")  
print(result.stdout)
```

Output of 'ls':
051-fix-my-code.ipynb
application.log
hello_world.txt
worldquant.txt

Now is your opportunity to use `subprocess.run` to capture the output of a command.

Task 5.1.3: Write code to capture the contents of "worldquant.txt", use the `subprocess.run` command.

```
In [40]: filename = "worldquant.txt"  
result = subprocess.run(["cat", filename], capture_output=True, text=True)  
  
print(f"Contents of '{filename}':")  
print(result.stdout)
```

Contents of 'worldquant.txt':
Hello WorldQuant!

In summary, we explored two methods for interacting with the command line in Python: `os.system` and `subprocess.run`. `os.system` works well for commands that just need to be executed. The resulting exit status codes indicate whether the command was successful or encountered errors. `subprocess.run` not only executes commands but also captures their output, allowing you to further use the results.

© 2024 by [WorldQuant University](#)